

ORG DIGITL COMP LAB
EECS 112L

Lab 4: Pipeline Processor

Joshua Ko
40786254
Nov 25, 2025

I. Objective

The objective is to create a pipelined processor using five different pipeline stages: Instruction Fetch, Instruction Decode, Execution, Memory, and Write Back.

Figure 1: Pipeline processor

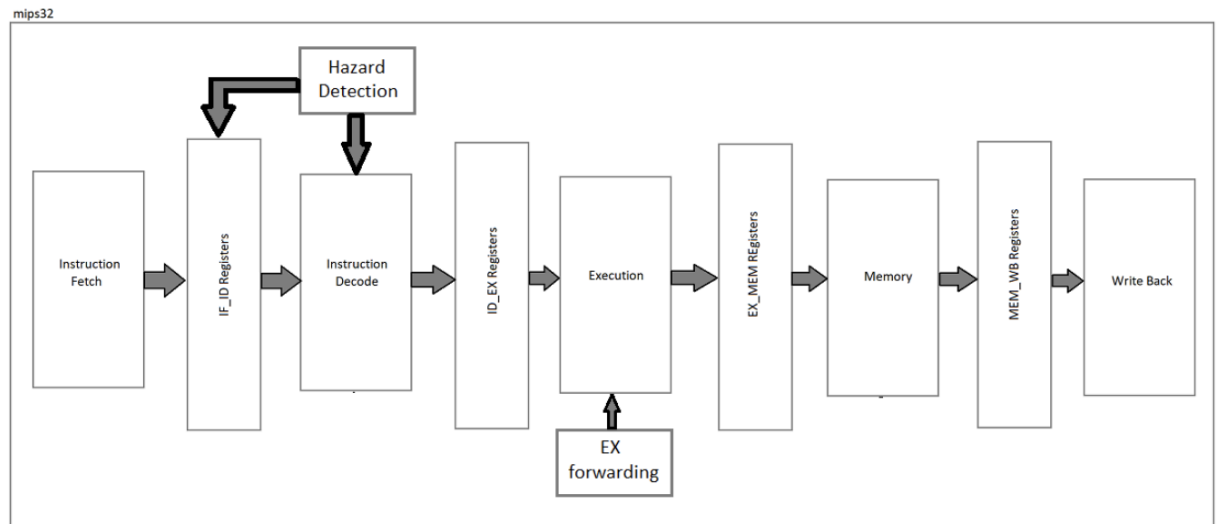


Figure 1: Block diagram of MIPS pipeline processor

II. Procedure

Review of Pipeline Architecture

The lab began with an examination of the five-stage MIPS pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). The purpose of this step was to understand how each stage operates independently and how pipeline registers are placed between stages to store intermediate results and control signals.

Identification of Pipeline Operations

For each instruction type, the flow of data and control signals through the pipeline was traced. This included determining which hardware components are active in each stage and how instructions advance from one pipeline register to the next on every clock cycle.

Analysis of Hazards

The three classes of pipeline hazards—structural, data, and control—were studied. In this step, example instruction sequences were used to identify where hazards arise when consecutive instructions require the same resources or depend on values not yet produced.

Implementation of Forwarding Paths

To mitigate data hazards, forwarding (bypassing) mechanisms were added to the pipeline. This involved:

- Designing and incorporating a **forwarding unit** capable of detecting data dependencies between pipeline stages.
- Adding the necessary multiplexers and data paths to enable values produced in later stages (EX, MEM, or WB) to be routed back to earlier stages without waiting for normal register writing.

Functionality was verified by stepping through sample instruction sequences and confirming that hazards were resolved without requiring stalls when forwarding was possible.

Design of the Hazard Detection Unit

For cases where forwarding could not resolve hazards—particularly load-use hazards—a **hazard detection unit** was implemented. This unit monitored the pipeline for situations in which the next instruction required data that could not be forwarded in time. When such conditions were detected, the unit inserted stall cycles by:

- Freezing appropriate pipeline registers,
- Preventing PC updates, and
- Injecting a bubble (NOP) into the pipeline.

Verification of Pipeline Behavior

After implementing forwarding and hazard detection logic, the modified pipeline was tested using representative instruction sequences. The goal was to verify that: Instructions flowed correctly through all five stages,

- Forwarding paths supplied needed operands without unnecessary delays, and
- Stalls were introduced only when required by unavoidable hazards.

Documentation and Analysis

Finally, results were recorded, including timing diagrams, hazard scenarios, and the behavior of forwarding and stall mechanisms. Observed performance improvements from pipelining were compared to the single-cycle execution model.

III. Simulation Results

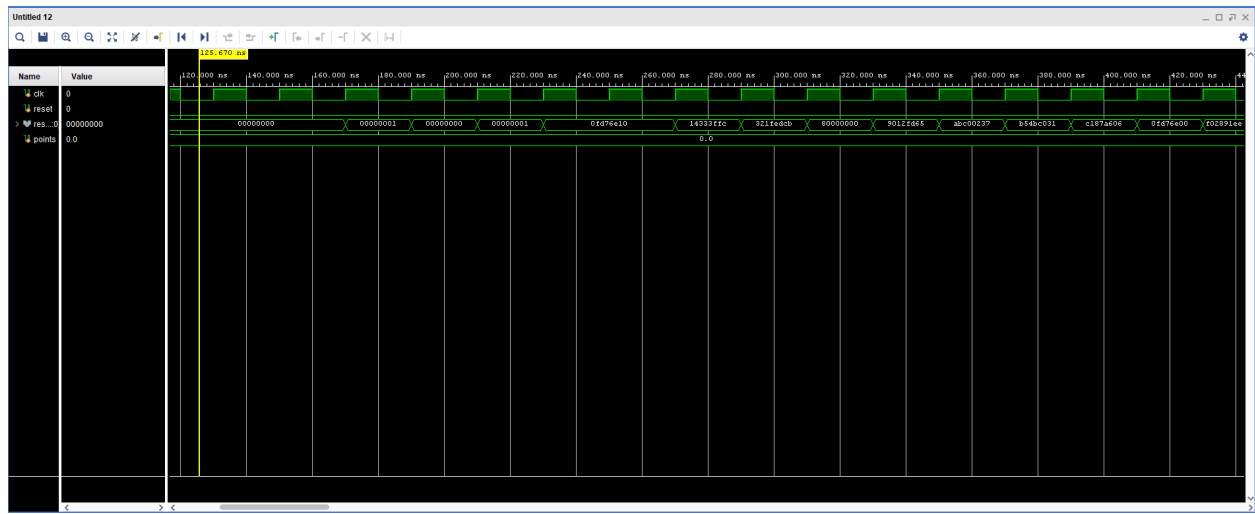


Figure 2: Screenshot of testbench results from beginning of the simulation



Figure 3: Screenshot of testbench results from end of the simulation

IF_pipe_stage.v

```
`timescale 1ns / 1ps

module IF_pipe_stage (
    input clk, reset,
    input en, // comes from data hazard
    input [9:0] branch_address,
    input [9:0] jump_address,
```

```

input branch_taken, // branch enable for mux
input jump, // jump enable for mux
output reg [9:0] pc_plus4,
output [31:0] instr
);

wire [9:0] branch_mux_output;
wire [9:0] jump_mux_output;
reg [9:0] pc;

// Branch mux instantiation
mux #(.mux_width(10)) branch_mux (
    .a(pc_plus4),
    .b(branch_address),
    .sel(branch_taken),
    .y(branch_mux_output));

// Jump mux instantiation
mux #(.mux_width(10)) jump_mux (
    .a(branch_mux_output),
    .b(jump_address),
    .sel(jump),
    .y(jump_mux_output));

instruction_mem inst_mem (
    .read_addr(jump_mux_output),
    .data(instr));

// Program counter (pc) register
always @(posedge clk or posedge reset) begin
    if (reset) begin
        pc_plus4 <= 10'd0; // Reset PC to 0
    end else if (en) begin
        pc_plus4 <= jump_mux_output + 10'd4; // Increment PC by 4
    end
end

// Register for actual PC value (keeping track of the current PC)
always @(posedge clk or posedge reset) begin
    if (reset) begin
        pc <= 10'd0; // Initialize PC to 0 on reset
    end else if (en) begin

```

```

        pc <= jump_mux_output; // Update PC based on the jump mux
output
    end
    end
endmodule

```

pipe_reg.v

```

`timescale 1ns / 1ps

module pipe_reg #( parameter WIDTH = 8)(
    input clk , reset ,
    input [ WIDTH -1:0] d ,
    output reg [ WIDTH -1:0] q
);

    always @ ( posedge clk or posedge reset )
    begin
        if ( reset )
            q <= 0;
        else
            q <= d ;
    end
endmodule

```

pipe_reg_en.v

```

`timescale 1ns / 1ps

module pipe_reg_en #(parameter WIDTH = 8)(
    input clk , reset ,
    input en , flush ,
    input [ WIDTH -1:0] d ,
    output reg [ WIDTH -1:0] q
);

    always @ ( posedge clk or posedge reset )
    begin
        if ( reset )
            q <= 0;
        else if ( flush )
            q <= 0;
    end
endmodule

```

```
        else if ( en )
            q <= d ;
    end
endmodule
```

IF_pipe_stage.v

```
`timescale 1ns / 1ps

module ID_pipe_stage (
    input clk , reset ,
    input [9:0] pc_plus4 ,
    input [31:0] instr ,
    input mem_wb_reg_write ,
    input [4:0] mem_wb_write_reg_addr ,
    input [31:0] mem_wb_write_back_data ,
    input Data_Hazard ,
    input Control_Hazard ,
    output [31:0] reg1 , reg2 , // done
    output [31:0] imm_value , // done
    output [9:0] branch_address , // done
    output [9:0] jump_address , // done
    output branch_taken , // done
    output [4:0] destination_reg , // done
    output mem_to_reg , // done
    output [1:0] alu_op , // done
    output mem_read , // done
    output mem_write , // done
    output alu_src , // done
    output reg_write , // done
    output jump // done
);

    wire reg_dst;
    wire branch;
    wire eq_test_out;
    wire control_mux_sel;
    wire mem_to_reg_mux;
    wire [1:0] alu_op_mux;
    wire mem_read_mux;
    wire mem_write_mux;
    wire alu_src_mux;
```

```

wire reg_write_mux;

assign branch_address = pc_plus4 + (imm_value << 2);
assign jump_address = instr[25:0] << 2;
assign eq_test_out = ((reg1 ^ reg2) == 32'b0) ? 1'b1 : 1'b0;
assign branch_taken = eq_test_out & branch;
assign control_mux_sel = ~Data_Hazard | Control_Hazard;

register_file reg_file_id (
    .clk(clk),
    .reset(reset),
    .reg_write_en(mem_wb_reg_write),
    .reg_write_dest(mem_wb_write_reg_addr),
    .reg_write_data(mem_wb_write_back_data),
    .reg_read_addr_1(instr[25:21]),
    .reg_read_addr_2(instr[20:16]),
    .reg_read_data_1(reg1),
    .reg_read_data_2(reg2));

sign_extend sign_ex_id (
    .sign_ex_in(instr[15:0]),
    .sign_ex_out(imm_value));

control control_unit_id(
    .reset(reset),
    .opcode(instr[31:26]),
    .reg_dst(reg_dst),
    .mem_to_reg(mem_to_reg_mux),
    .alu_op(alu_op_mux),
    .mem_read(mem_read_mux),
    .mem_write(mem_write_mux),
    .alu_src(alu_src_mux),
    .reg_write(reg_write_mux),
    .branch(branch),
    .jump(jump));

mux #(.mux_width(5)) reg_dst_mux (
    .a(instr[20:16]),
    .b(instr[15:11]),
    .sel(reg_dst),
    .y(destination_reg));

```



```

mux #(.mux_width(1)) ctrl_mem_to_reg_mux (
    .a(mem_to_reg_mux),
    .b(1'b0),
    .sel(control_mux_sel),
    .y(mem_to_reg));

mux #(.mux_width(2)) ctrl_alu_op_mux (
    .a(alu_op_mux),
    .b(2'd0),
    .sel(control_mux_sel),
    .y(alu_op));

mux #(.mux_width(1)) ctrl_mem_read_mux (
    .a(mem_read_mux),
    .b(1'b0),
    .sel(control_mux_sel),
    .y(mem_read));

mux #(.mux_width(1)) ctrl_mem_write_mux (
    .a(mem_write_mux),
    .b(1'b0),
    .sel(control_mux_sel),
    .y(mem_write));

mux #(.mux_width(1)) ctrl_alu_src_mux (
    .a(alu_src_mux),
    .b(1'b0),
    .sel(control_mux_sel),
    .y(alu_src));

mux #(.mux_width(1)) ctrl_reg_write_mux (
    .a(reg_write_mux),
    .b(1'b0),
    .sel(control_mux_sel),
    .y(reg_write));

endmodule

```

ID_pipe_stage.v

```

`timescale 1ns / 1ps

```

```

module IF_pipe_stage (
    input clk, reset,
    input en, // comes from data hazard
    input [9:0] branch_address,
    input [9:0] jump_address,
    input branch_taken, // branch enable for mux
    input jump, // jump enable for mux
    output reg [9:0] pc_plus4,
    output [31:0] instr
);

wire [9:0] branch_mux_output;
wire [9:0] jump_mux_output;
reg [9:0] pc;

// Branch mux instantiation
mux #(.mux_width(10)) branch_mux (
    .a(pc_plus4),
    .b(branch_address),
    .sel(branch_taken),
    .y(branch_mux_output));

// Jump mux instantiation
mux #(.mux_width(10)) jump_mux (
    .a(branch_mux_output),
    .b(jump_address),
    .sel(jump),
    .y(jump_mux_output));

instruction_mem inst_mem (
    .read_addr(jump_mux_output),
    .data(instr));

// Program counter (pc) register
always @(posedge clk or posedge reset) begin
    if (reset) begin
        pc_plus4 <= 10'd0; // Reset PC to 0
    end else if (en) begin
        pc_plus4 <= jump_mux_output + 10'd4; // Increment PC by 4
    end
end
end

```

```

        // Register for actual PC value (keeping track of the current PC)
        always @(posedge clk or posedge reset) begin
            if (reset) begin
                pc <= 10'd0; // Initialize PC to 0 on reset
            end else if (en) begin
                pc <= jump_mux_output; // Update PC based on the jump mux
            end
        end
    end

endmodule

```

EX_pipe_stage.v

```

`timescale 1ns / 1ps

module EX_pipe_stage (
    input [31:0] id_ex_instr ,
    input [31:0] reg1 , reg2 ,
    input [31:0] id_ex_imm_value ,
    input [31:0] ex_mem_alu_result ,
    input [31:0] mem_wb_write_back_result ,
    input id_ex_alu_src ,
    input [1:0] id_ex_alu_op ,
    input [1:0] Forward_A , Forward_B ,
    output [31:0] alu_in2_out ,
    output [31:0] alu_result
);

    wire [31:0] forward_a_out;
    wire [31:0] forward_b_out;

    // Forward A (for reg1)
    mux4 #(.mux_width(32)) id_ex_reg1_mux (
        .a(reg1),
        .b(mem_wb_write_back_result),
        .c(ex_mem_alu_result),
        .d(),
        .sel(Forward_A),
        .y(forward_a_out));

    // Forward B (for reg2)

```

```

mux4 #(.mux_width(32)) id_ex_reg2_mux (
    .a(reg2),
    .b(mem_wb_write_back_result),
    .c(ex_mem_alu_result),
    .d(),
    .sel(Forward_B),
    .y(forward_b_out));

// ALU second input mux
mux #(.mux_width(32)) id_ex_alu_mux (
    .a(forward_b_out),
    .b(id_ex_imm_value),
    .sel(id_ex_alu_src),
    .y(alu_in2_out));

// ALU control unit
wire [5:0] funct = id_ex_instr[5:0];
wire [3:0] ALU_Control;

ALUControl alu_ctrl_unit (
    .ALUOp(id_ex_alu_op),
    .Function(funct),
    .ALU_Control(ALU_Control)
);

// ALU execution
ALU alu_inst (
    .a(forward_a_out),
    .b(alu_in2_out),
    .alu_control(ALU_Control),
    .zero(), // Not used here
    .alu_result(alu_result)
);
endmodule

```

EX_Forwarding_unit.v

```

`timescale 1ns / 1ps

module EX_Forwarding_unit (
    input ex_mem_reg_write,
    input [4:0] ex_mem_write_reg_addr,
    input [4:0] id_ex_instr_rs,

```

```

input [4:0] id_ex_instr_rt,
input mem_wb_reg_write,
input [4:0] mem_wb_write_reg_addr,
output reg [1:0] Forward_A,
output reg [1:0] Forward_B
);

always @(*) begin
    // Default: no forwarding
    Forward_A = 2'b00;
    Forward_B = 2'b00;

    // 1a. EX hazard: EX/MEM.RegisterRd == ID/EX.RegisterRs
    if (ex_mem_reg_write &&
        (ex_mem_write_reg_addr != 5'd0) &&
        (ex_mem_write_reg_addr == id_ex_instr_rs))
    begin
        Forward_A = 2'b10; // Forward from EX/MEM
    end

    // 2a. MEM hazard: MEM/WB.RegisterRd == ID/EX.RegisterRs
    else if (mem_wb_reg_write &&
        (mem_wb_write_reg_addr != 5'd0) &&
        (mem_wb_write_reg_addr == id_ex_instr_rs))
    begin
        Forward_A = 2'b01; // Forward from MEM/WB
    end

    // 1b. EX hazard: EX/MEM.RegisterRd == ID/EX.RegisterRt
    if (ex_mem_reg_write &&
        (ex_mem_write_reg_addr != 5'd0) &&
        (ex_mem_write_reg_addr == id_ex_instr_rt))
    begin
        Forward_B = 2'b10; // Forward from EX/MEM
    end

    // 2b. MEM hazard: MEM/WB.RegisterRd == ID/EX.RegisterRt
    else if (mem_wb_reg_write &&
        (mem_wb_write_reg_addr != 5'd0) &&
        (mem_wb_write_reg_addr == id_ex_instr_rt))
    begin
        Forward_B = 2'b01; // Forward from MEM/WB
    end
end

```

```

        end

    end

endmodule

```

Hazard_detection.v

```

`timescale 1ns / 1ps

module Hazard_detection (
    input id_ex_mem_read ,
    input [4:0] id_ex_destination_reg ,
    input [4:0] if_id_rs , if_id_rt ,
    input branch_taken , jump ,
    output reg Data_Hazard ,
    output reg IF_Flush
);
always @ (*)
begin
    if (( id_ex_mem_read == 1'b1 ) &
        (( id_ex_destination_reg == if_id_rs ) | ( id_ex_destination_reg ==
if_id_rt )) )
        Data_Hazard = 1'b0 ;
    else
        Data_Hazard = 1'b1 ;
        IF_Flush = branch_taken | jump ;
    end
endmodule

```

data_memory.v

```

`timescale 1ns / 1ps

module data_memory(
    input clk,
    input [31:0] mem_access_addr,
    input [31:0] mem_write_data,
    input mem_write_en,
    input mem_read_en,
    output [31:0] mem_read_data);

```

```

integer i;
reg [31:0] ram [255:0];
wire [7 : 0] ram_addr = mem_access_addr[9 : 2];

initial
begin
    for(i=0;i<256;i=i+1)
        ram[i] <= 32'd0;
    end
    always @(posedge clk) begin
        if (mem_write_en)
            ram[ram_addr] <= mem_write_data;
        end

    assign mem_read_data = (mem_read_en==1'b1) ? ram[ram_addr]: 32'd0;

endmodule

```

mips_32.v

```

`timescale 1ns / 1ps

module mips_32(
    input clk, reset,
    output[31:0] result
);

// define all the wires here. You need to define more wires than the ones
// you did in Lab2
// Program Counter wires
reg [9:0] pc;
wire [9:0] pc_plus4, next_pc;

// IF wires
wire [31:0] instr;

// IF/ID pipe reg wires
wire [31:0] if_id_instr;
wire [9:0] if_id_pc_plus4;

```

```

// ID wires
wire [31:0] id_reg1, id_reg2, id_imm_value;
wire [9:0] id_branch_address, id_jump_address;
wire id_branch_taken;
wire [4:0] id_destination_reg;
wire id_mem_to_reg, id_mem_read, id_mem_write;
wire id_alu_src, id_reg_write, id_jump;
wire [1:0] id_alu_op;
wire Data_Hazard;
wire IF_Flush;

// ID/EX pipe reg wires
wire [31:0] id_ex_instr;
wire [31:0] id_ex_reg1, id_ex_reg2, id_ex_imm_value;
wire [4:0] id_ex_destination_reg;
wire id_ex_alu_src, id_ex_mem_read, id_ex_mem_write;
wire id_ex_mem_to_reg, id_ex_reg_write, id_ex_jump;
wire [3:0] id_ex_alu_op;
wire [9:0] id_ex_branch_address, id_ex_jump_address;
wire id_ex_branch_taken;

// EX wires
wire [31:0] alu_in2_out;
wire [31:0] alu_result;
wire [1:0] Forward_A, Forward_B;

// EX/MEM pipe reg wires
wire [31:0] ex_mem_alu_result, ex_mem_reg2;
wire [4:0] ex_mem_destination_reg;
wire ex_mem_mem_to_reg, ex_mem_mem_read, ex_mem_mem_write,
ex_mem_reg_write;

// MEM wires
wire [31:0] mem_read_data;

// MEM/WB pipe reg wires
wire [31:0] mem_wb_read_data, mem_wb_alu_result;
wire mem_wb_mem_to_reg, mem_wb_reg_write;
wire [4:0] mem_wb_write_reg_addr;

// WB wires
wire [31:0] mem_wb_write_back_data;

```



```

// branch and jump wires
wire [9:0] branch_pc;
wire [9:0] jump_pc;
// Build the pipeline as indicated in the lab manual

// Program Counter
always @(posedge clk or posedge reset) begin
    if (reset)
        pc <= 10'b0;
    else
        pc <= next_pc;
end

assign pc_plus4 = pc + 10'd4;

//////////////////// Instruction Fetch
IF_pipe_stage if_stage (
    .clk(clk),
    .reset(reset),
    .en(Data_Hazard),           // stall if hazard
    .branch_address(id_branch_address),
    .jump_address(id_jump_address),
    .branch_taken(id_branch_taken),
    .jump(id_jump),
    .pc_plus4(if_id_pc_plus4),   // output
    .instr(if_id_instr)         // output
);

//////////////////// IF/ID registers
pipe_reg_en #(.WIDTH(32)) if_id_instr_reg (
    .clk(clk), .reset(reset),
    .en(Data_Hazard),           // Stall IF/ID if hazard
    .flush(IF_Flush),           // Flush on branch/jump
    .d(instr),
    .q(if_id_instr)
);

pipe_reg_en #(.WIDTH(10)) if_id_pc_reg (
    .clk(clk), .reset(reset),
    .en(Data_Hazard),
    .flush(IF_Flush),

```

```

        .d(pc_plus4),
        .q(if_id_pc_plus4)
    );

////////// Instruction Decode
ID_pipe_stage id_stage (
    .clk(clk),
    .reset(reset),

    .pc_plus4(if_id_pc_plus4),
    .instr(if_id_instr),

    .mem_wb_reg_write(mem_wb_reg_write),
    .mem_wb_write_reg_addr(mem_wb_write_reg_addr),
    .mem_wb_write_back_data(mem_wb_write_back_data),

    .Data_Hazard(Data_Hazard),
    .Control_Hazard(IF_Flush),

    .reg1(id_reg1),
    .reg2(id_reg2),
    .imm_value(id_imm_value),
    .branch_address(id_branch_address),
    .jump_address(id_jump_address),
    .branch_taken(id_branch_taken),
    .destination_reg(id_destination_reg),
    .mem_to_reg(id_mem_to_reg),
    .alu_op(id_alu_op),
    .mem_read(id_mem_read),
    .mem_write(id_mem_write),
    .alu_src(id_alu_src),
    .reg_write(id_reg_write),
    .jump(id_jump)
);

////////// ID/EX registers
pipe_reg #(.WIDTH(32)) pr_id_ex_instr
(.clk(clk),.reset(reset),.d(if_id_instr),          .q(id_ex_instr));

```

```

pipe_reg #(.WIDTH(32)) pr_id_ex_reg1
(.clk(clk),.reset(reset),.d(id_reg1),          .q(id_ex_reg1));
pipe_reg #(.WIDTH(32)) pr_id_ex_reg2
(.clk(clk),.reset(reset),.d(id_reg2),          .q(id_ex_reg2));
pipe_reg #(.WIDTH(32)) pr_id_ex_imm
(.clk(clk),.reset(reset),.d(id_imm_value),      .q(id_ex_imm_value));
pipe_reg #(.WIDTH(5 )) pr_id_ex_dest
(.clk(clk),.reset(reset),.d(id_destination_reg),
.q(id_ex_destination_reg));
pipe_reg #(.WIDTH(1 )) pr_id_ex_alu_src
(.clk(clk),.reset(reset),.d(id_alu_src),        .q(id_ex_alu_src));
pipe_reg #(.WIDTH(2 )) pr_id_ex_alu_op
(.clk(clk),.reset(reset),.d(id_alu_op),        .q(id_ex_alu_op));
pipe_reg #(.WIDTH(1 )) pr_id_ex_mem_read
(.clk(clk),.reset(reset),.d(id_mem_read),      .q(id_ex_mem_read));
pipe_reg #(.WIDTH(1 )) pr_id_ex_mem_write
(.clk(clk),.reset(reset),.d(id_mem_write),     .q(id_ex_mem_write));
pipe_reg #(.WIDTH(1 )) pr_id_ex_mem_to_reg
(.clk(clk),.reset(reset),.d(id_mem_to_reg),    .q(id_ex_mem_to_reg));
pipe_reg #(.WIDTH(1 )) pr_id_ex_reg_write
(.clk(clk),.reset(reset),.d(id_reg_write),     .q(id_ex_reg_write));
pipe_reg #(.WIDTH(10)) pr_id_ex_branch_addr
(.clk(clk),.reset(reset),.d(id_branch_address), .q(id_ex_branch_address));
pipe_reg #(.WIDTH(1 )) pr_id_ex_branch_taken
(.clk(clk),.reset(reset),.d(id_branch_taken),  .q(id_ex_branch_taken));
pipe_reg #(.WIDTH(10)) pr_id_ex_jump_addr
(.clk(clk),.reset(reset),.d(id_jump_address),  .q(id_ex_jump_address));
pipe_reg #(.WIDTH(1 )) pr_id_ex_jump
(.clk(clk),.reset(reset),.d(id_jump),          .q(id_ex_jump));

////////// Hazard_detection unit
Hazard_detection hazard_unit (
    .id_ex_mem_read(id_ex_mem_read),
    .id_ex_destination_reg(id_ex_destination_reg),
    .if_id_rs(if_id_instr[25:21]),
    .if_id_rt(if_id_instr[20:16]),
    .branch_taken(id_branch_taken),
    .jump(id_jump),
    .Data_Hazard(Data_Hazard),
    .IF_Flush(IF_Flush)
);

```

```
//////////////////////////////////// Execution
```

```
EX_pipe_stage ex_stage (  
    .id_ex_instr(id_ex_instr),  
    .reg1(id_ex_reg1),  
    .reg2(id_ex_reg2),  
    .id_ex_imm_value(id_ex_imm_value),  
    .ex_mem_alu_result(ex_mem_alu_result),  
    .mem_wb_write_back_result(mem_wb_write_back_data),  
    .id_ex_alu_src(id_ex_alu_src),  
    .id_ex_alu_op(id_ex_alu_op),  
    .Forward_A(Forward_A),  
    .Forward_B(Forward_B),  
    .alu_in2_out(alu_in2_out),  
    .alu_result(alu_result)  
);
```

```
//////////////////////////////////// Forwarding unit
```

```
EX_Forwarding_unit forwarding_unit (  
    .ex_mem_reg_write(ex_mem_reg_write),  
    .ex_mem_write_reg_addr(ex_mem_write_reg_addr),  
    .id_ex_instr_rs(id_ex_instr[25:21]),  
    .id_ex_instr_rt(id_ex_instr[20:16]),  
    .mem_wb_reg_write(mem_wb_reg_write),  
    .mem_wb_write_reg_addr(mem_wb_write_reg_addr),  
    .Forward_A(Forward_A),  
    .Forward_B(Forward_B)  
);
```

```
//////////////////////////////////// EX/MEM registers
```

```
pipe_reg #(.WIDTH(32)) pr_ex_mem_alu_result  
(.clk(clk),.reset(reset),.d(alu_result), .q(ex_mem_alu_result));  
pipe_reg #(.WIDTH(32)) pr_ex_mem_reg2  
(.clk(clk),.reset(reset),.d(id_ex_reg2), .q(ex_mem_reg2));  
pipe_reg #(.WIDTH(5 )) pr_ex_mem_dest  
(.clk(clk),.reset(reset),.d(id_ex_destination_reg),  
.q(ex_mem_destination_reg));  
pipe_reg #(.WIDTH(1 )) pr_ex_mem_m2r  
(.clk(clk),.reset(reset),.d(id_ex_mem_to_reg), .q(ex_mem_mem_to_reg));
```

```

pipe_reg #(.WIDTH(1 )) pr_ex_mem_mread
(.clk(clk),.reset(reset),.d(id_ex_mem_read),    .q(ex_mem_mem_read));
pipe_reg #(.WIDTH(1 )) pr_ex_mem_mwrite
(.clk(clk),.reset(reset),.d(id_ex_mem_write),    .q(ex_mem_mem_write));
pipe_reg #(.WIDTH(1 )) pr_ex_mem_regwrite
(.clk(clk),.reset(reset),.d(id_ex_reg_write),    .q(ex_mem_reg_write));

//////////////////// memory
data_memory data_mem (
    .clk(clk),
    .mem_access_addr(ex_mem_alu_result),
    .mem_write_data(ex_mem_reg2),
    .mem_write_en(ex_mem_mem_write),
    .mem_read_en(ex_mem_mem_read),
    .mem_read_data(mem_read_data)
);

//////////////////// MEM/WB registers
pipe_reg #(.WIDTH(32)) pr_mem_wb_read_data
(.clk(clk),.reset(reset),.d(mem_read_data),        .q(mem_wb_read_data));
pipe_reg #(.WIDTH(32)) pr_mem_wb_alu
(.clk(clk),.reset(reset),.d(ex_mem_alu_result),    .q(mem_wb_alu_result));
pipe_reg #(.WIDTH(5 )) pr_mem_wb_dest
(.clk(clk),.reset(reset),.d(ex_mem_destination_reg),.q(mem_wb_write_reg_addr));
pipe_reg #(.WIDTH(1 )) pr_mem_wb_m2r
(.clk(clk),.reset(reset),.d(ex_mem_mem_to_reg),    .q(mem_wb_mem_to_reg));
pipe_reg #(.WIDTH(1 )) pr_mem_wb_regwrite
(.clk(clk),.reset(reset),.d(ex_mem_reg_write),    .q(mem_wb_reg_write));

//////////////////// writeback
mux #(.mux_width(32)) wb_mux (
    .a(mem_wb_alu_result),
    .b(mem_wb_read_data),
    .sel(mem_wb_mem_to_reg),
    .y(mem_wb_write_back_data)
);

```

```
assign result = mem_wb_write_back_data;
```

```
endmodule
```